Guida passo-passo: robot a due ruote in equilibrio

Pipeline standard NVIDIA (FreeCAD → URDF → Isaac Lab → RL)

Progetto robot_slalom

Aggiornata al 24 ottobre 2026

Scopo di questa guida

Documenta, passo per passo e in ordine cronologico, la pipeline standard NVIDIA/Isaac Lab usata in questo progetto: dal modello CAD fino all'addestramento con reinforcement learning. Viene aggiornata ad ogni passo compiuto. La regola generale della pipeline (cosa è "standard" e cosa no) è in `PIPELINE_STANDARD_NVIDIA.md`.

Indice

1 Punto di partenza: dal CAD Fusion a FreeCAD **1**

2 Installazione FreeCAD e del server MCP **1**

3 Installazione del workbench CROSS (export URDF) **2**

4 I concetti: Robot, Link, Joint **3**

5 Composizione del robot in CROSS **3**

5.1 Struttura 3

5.2 Pezzi CAD in ogni link 4

5.3 Giunti 4

5.4 Come si crea dall'editor 4

5.5 Problemi incontrati 5

6 Prossimi passi (da fare) **5**

1 Punto di partenza: dal CAD Fusion a FreeCAD

Il modello del robot a due ruote è stato progettato in **Fusion 360** (CAD): corpo, motori, ruote, batteria, elettronica (ESP32-S3, sensori ToF, camera). Da qui si parte seguendo alla lettera la pipeline standard NVIDIA, usando **FreeCAD** al posto di Fusion 360 per il passaggio successivo, perché ha un esportatore URDF nativo tramite il workbench **CROSS**:

1. **Importazione**: il modello viene esportato da Fusion in formato neutro (`STEP`, file `export\robot_2ruote_esp32_varD.step`) e importato in FreeCAD.
2. **CROSS**: dentro FreeCAD si definiscono i link e i giunti del robot (corpo, ruota sx, ruota dx, giunti rivoluti attorno all'asse delle ruote) e si assegnano materiali/densità ai pezzi.
3. **Esportazione**: CROSS esporta direttamente `robot.urdf`, con masse, inerzie e mesh incluse.
4. **Simulazione**: Isaac Sim importa l'URDF (`sim_utils.UrdfFileCfg`) e lo converte in USD.

5. **Isaac Lab**: `ArticulationCfg` carica il robot, `DirectRLEnv` definisce l'ambiente RL, `rsl_rl` esegue l'addestramento PPO.

Tutto il codice Python di supporto (configurazione del robot, ambiente RL, script di training, visualizzazione e valutazione) verrà scritto da zero a partire dal nuovo URDF, seguendo solo l'API standard di Isaac Lab, senza bridge o file intermedi scritti a mano.

2 Installazione FreeCAD e del server MCP

Data: 24 ottobre 2026.

1. FreeCAD 1.1 installato in `C:\Program Files\FreeCAD 1.1`.

2. Server MCP `freecad-robust-mcp` (progetto spkane/freecad-addon-robust-mcp-server) installato con pip (Python 3.11, richiesto per compatibilità ABI con FreeCAD):

```
python -m pip install --user freecad-robust-mcp
```

3. Registrato in Claude Code:

```
claude mcp add freecad "C:\Users\aless\AppData\Roaming\Python\
   Python311\Scripts\freecad-mcp.exe" -e FREECAD_MODE=xmlrpc
```

4. Il server richiede la libreria `mcp` versione 1.x (con la 2.x si chiude all'avvio):

```
python -m pip install --user "mcp<2"
```

5. **Workbench dentro FreeCAD**: *Robust MCP Bridge* non è nel catalogo dell'Addon Manager, quindi va installato a mano. Dal repository si copia la cartella `freecad\RobustMCPBridge` in

```
%APPDATA%\FreeCAD\v1-1\Mod\RobustMCPBridge\freecad\RobustMCPBridge\
```

   e si mette un `package.xml` in `...\Mod\RobustMCPBridge\`. Attenzione: FreeCAD 1.1 legge gli addon dalla cartella con la versione (`v1-1\Mod`), non da `FreeCAD\Mod`.

6. Il Python interno di FreeCAD deve avere le dipendenze del bridge (mancava `anyio`):

```
"C:\Program␣Files\FreeCAD␣1.1\bin\python.exe" -m pip install --user
   freecad-robust-mcp
```

7. Avvio: in FreeCAD selezionare il workbench *Robust MCP Bridge* e cliccare *Start MCP Bridge*. Nel Report view compare "MCP Bridge started!" (XML-RPC 9875, socket 9876) e `claude mcp list` mostra `freecad: Connected`.

8. Avvio automatico del bridge a ogni apertura di FreeCAD: in `%APPDATA%\FreeCAD\v1-1\user.cfg`, nel gruppo `RobustMCPBridge`, aggiungere `<FCBool Name="AutoStart" Value="1"/>` (con FreeCAD chiuso, altrimenti viene sovrascritto).

3 Installazione del workbench CROSS (export URDF)

CROSS non è nel catalogo standard dell'Addon Manager (cercando "cross" escono solo *Cross Section* e *Mouse Cross*). Si installa a mano dal repository `github.com/galou/freecad.cross`:

1. Clonare il repository nella cartella Mod, **con il nome esatto** `freecad.cross` (il codice cerca icone, template e finestre in `Mod\freecad.cross\resources`: con un altro nome il workbench si carica ma i template dell'export non vengono trovati):

```
cd %APPDATA%\FreeCAD\v1-1\Mod
git clone --depth 1 https://github.com/galou/freecad.cross.git freecad
    .cross
```

2. Creare `Mod\freecad.cross\InitGui.py` con una sola riga (l'Addon Manager lo farebbe da sé; col clone manuale va scritto):

```
import freecad.cross.init_gui
```

3. Installare le dipendenze dichiarate in `package.xml` nel Python di FreeCAD:

```
"C:\Program␣Files\FreeCAD␣1.1\bin\python.exe" -m pip install --user
    urdf-parser-py xacro
```

4. Riavviare FreeCAD. Nell'elenco dei workbench compare *CROSS - ROS workbench* con i comandi NewRobot, NewLink, NewJoint, SetMaterial, CalculateMassAndInertia, UrdfExport. L'avviso "`ROS_DISTRO` is not set" è normale: ROS non serve per esportare l'URDF.

4 I concetti: Robot, Link, Joint

Un robot per la simulazione (formato URDF) si descrive con tre soli mattoni, che in CROSS corrispondono ai pulsanti *New Robot*, *New Link*, *New Joint*:

Link = un pezzo rigido. Un insieme di pezzi CAD che si muovono sempre insieme, come se fossero incollati. Per il simulatore è un unico oggetto con massa, baricentro e inerzia. Il link non ha una forma propria: è un'"etichetta" che raggruppa pezzi CAD (per questo nell'albero di FreeCAD, passandoci sopra col mouse, non si evidenzia niente).

Joint = il collegamento tra due link. Dice come un link si muove rispetto all'altro:

- **Parent**: il pezzo "fisso"; **Child**: il pezzo che si muove.
- **Type**: `continuous` (gira all'infinito, come una ruota), `revolute` (gira entro dei limiti, come un gomito), `prismatic` (scorre, come un cassetto), `fixed` (bloccato).
- **Origin**: posizione del perno e direzione dell'asse di rotazione.
- **Effort / Velocity**: coppia (N·m) e velocità (rad/s) massime del motore.

Robot = il contenitore che tiene insieme link e giunti; è ciò che si esporta come `robot.urdf`.

In Isaac Lab la rete neurale comanderà i **giunti** (quanta coppia dare a ogni ruota); il simulatore calcola come si muove il resto in base alle masse dei **link**.

5 Composizione del robot in CROSS

Data: 24 ottobre 2026. File: `export\robot_cross.FCStd` (recuperato dall'autosalvataggio di FreeCAD dopo uno spegnimento del PC, poi salvato con questo nome).

5.1 Struttura

Il robot è fatto di **3 link** e **2 joint**:

- **Link** (pezzi rigidi): `base_link` (il telaio), `ruota_sx` (ruota sinistra), `ruota_dx` (ruota destra). **Anche ogni ruota è un link**: è un pezzo rigido a sé, separato dal telaio, perché si muove (gira) rispetto ad esso.
- **Joint** (collegamenti): `giunto_ruota_sx` collega il telaio alla ruota sinistra, `giunto_ruota_dx` collega il telaio alla ruota destra.

```
              [LINK] base_link  (telaio, 393 g)
                /                       \
  [JOINT] giunto_ruota_sx       [JOINT] giunto_ruota_dx
       (continuous)                  (continuous)
             |                             |
     [LINK] ruota_sx               [LINK] ruota_dx
        (42 g)                        (42 g)
```

In generale: un link per ogni parte che si muove in modo indipendente, un joint tra ogni coppia di link collegati. Ogni joint ha sempre un link *parent* (sopra) e un link *child* (sotto).

Regola usata: tutto ciò che è avvitato al telaio (anche i motori, il cui corpo non gira) va in `base_link`; solo ciò che ruota con l'albero del motore va nel link della ruota.

5.2 Pezzi CAD in ogni link

Lo STEP importato da Fusion produce 40 solidi (`robot_2ruote_esp32_varD000...039`), rinominati con nomi leggibili. I nomi incerti (`componente_1`, `barra_o_cavo_1`, ...) sono pezzi non riconoscibili con sicurezza dal CAD.

`base_link` (36 pezzi, 0,393 kg, baricentro x=1,3, y=0,6, z=59,0 mm):

- Struttura: piastra_superiore, piastra_inferiore, colonnina_ant_sx/dx, colonnina_post_sx/dx, vite_o_distanziale_1/2, barra_o_cavo_1/2.
- Motori e staffe: motore_sx, motore_dx, staffa_motore_sx_sup/inf, staffa_motore_dx_sup/inf.
- Batteria: vano_batteria, cella_batteria_1/2.
- Elettronica: scheda_esp32s3, scheda_driver_motori, scheda_piccola_1/2, componente_1, connettore_o_componente_1/2.
- Camera: staffa_camera_1...5.
- Appoggi: supporto_anteriore_centro/sx/dx, piede_anteriore, piede_posteriore.

`ruota_sx` (0,042 kg, centro y=+88, z=32,5 mm): ruota_sx_corpo, ruota_sx_gomma.

`ruota_dx` (0,042 kg, centro y=−88, z=32,5 mm): ruota_dx_corpo, ruota_dx_gomma.

5.3 Giunti

Giunto	Tipo	Parent → Child	Origin (mm, rotazione)	Effort / Velocity
`giunto_ruota_sx`	continuous	base_link → ruota_sx	(0, +88, 32,5), roll −90°	0,30 N·m / 29,3 rad/s
`giunto_ruota_dx`	continuous	base_link → ruota_dx	(0, −88, 32,5), roll −90°	0,30 N·m / 29,3 rad/s

La rotazione di −90° porta l'asse z del giunto (asse di rotazione in URDF) lungo l'asse delle ruote. Effort e Velocity sono i valori **tipici, non misurati** del motore JGA25-370 a 6 V: coppia di stallo 0,30 N·m e 280 giri/min a vuoto ($280 \cdot 2\pi/60 = 29{,}3$ rad/s).

5.4 Come si crea dall'editor

1. Workbench *CROSS - ROS workbench*.
2. *New Robot* (`N`,`R`): crea il contenitore `robot`.
3. *New Link* (`N`,`L`) con il robot selezionato; rinominare (F2) in `base_link`. Nel pannello *Dati*, proprietà **Real** → "..." e spuntare i pezzi elencati sopra; **Visual** e **Collision** con gli stessi pezzi.
4. Stessa cosa per `ruota_sx` e `ruota_dx`.
5. *New Joint* (`N`,`J`): nel pannello Dati Type, Parent, Child, Origin, Effort, Velocity come in tabella. Usare *Set placement* per posizionare le mesh.
6. *Set the material to the whole robot or a link*, poi *Calculate mass and inertia* con il robot selezionato: riempie Mass e Ixx... Izz di ogni link.

5.5 Problemi incontrati

- **Si vedono solo frecce**: sui link `ShowVisual` e `ShowReal` (pannello *Vista*) erano `False`. Messi a `True`, e nascosti i 40 pezzi CAD originali per non vederli doppi.
- **Ruote staccate**: i pezzi CAD sono in coordinate assolute, ma il link applicava di nuovo la propria posizione (± 88 mm, $-90°$). Corretto impostando su `ruota_sx/dx` `MountedPlacement` = inverso del `Placement` del link (è ciò che fa *Set placement*).
- **Colori** per riconoscere i link: grigio `base_link`, rosso `ruota_sx`, blu `ruota_dx`. Le frecce blu al centro delle ruote sono gli assi dei giunti (`ShowAxis`, lunghezza 60 mm).
- **Materiale non assegnato**: `MaterialDensity` è vuoto e `MaterialNotCalculate = True`, quindi masse e inerzie sono state calcolate con una densità non documentata: la massa del telaio (393 g) va confrontata con quella del CAD Fusion.
- **Bridge MCP che non parte da solo**: all'avvio compare "`module 'FreeCAD' has no attribute 'GuiUp'`" e l'auto-start fallisce; avviarlo dal workbench *Robust MCP Bridge* con *Start MCP Bridge*.

6 Prossimi passi (da fare)

1. Verificare/assegnare materiali e densità e ricalcolare massa e inerzia (confronto con `STATO_robot_2ruote_RL.md`).
2. Esportare l'URDF da CROSS in `robot_sim\` (corpo attualmente vuota).
3. Scrivere da capo `robot_cfg.py` (ArticulationCfg/UrdfFileCfg) e l'ambiente RL (DirectRLEnv) usando i nomi di link/giunti del nuovo URDF.
4. Scrivere da capo gli script di training/visualizzazione/valutazione e confrontare i risultati con quelli precedenti (8,33° di appoggio, reward RL) per verificare l'equivalenza del nuovo modello.